

SHETH L.U.J AND SIR M.V COLLEGE

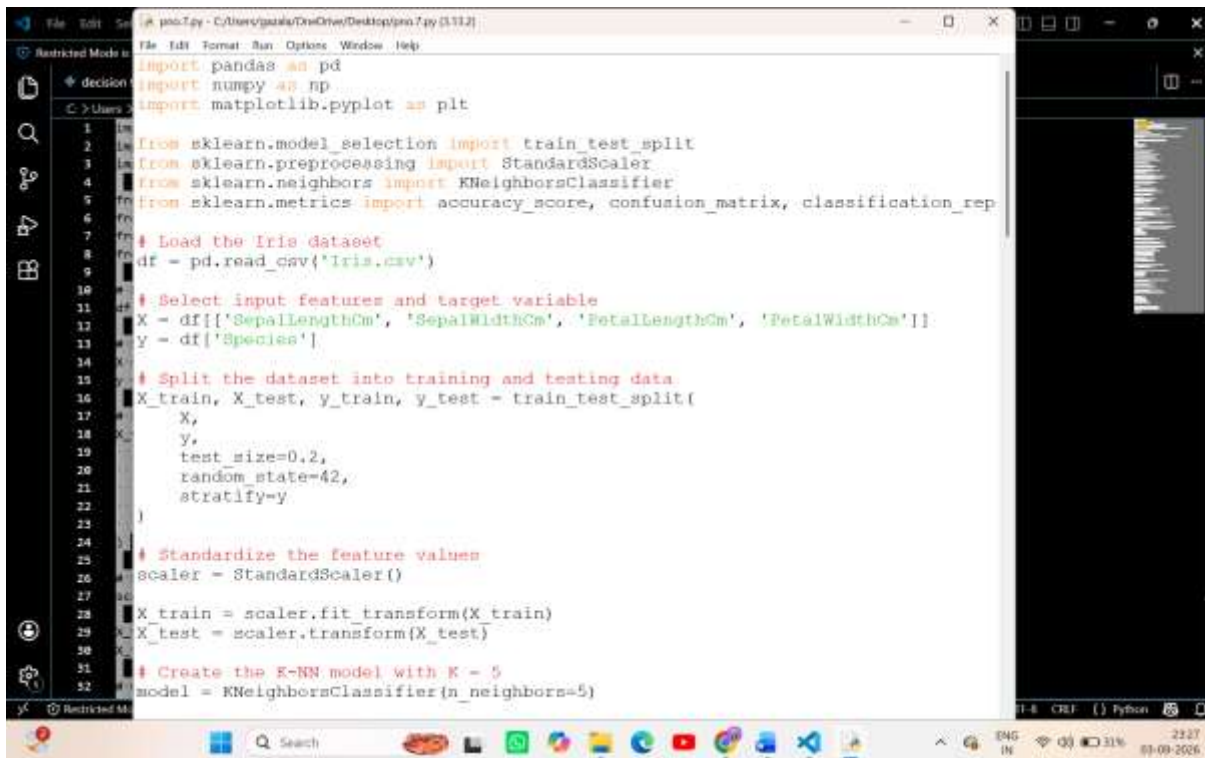
Artificial Intelligence

Practical No.7

Aim: K-Nearest Neighbors (K-NN)

- Implement the K-NN algorithm for classification or regression.
- Apply the K-NN algorithm to a given dataset and predict the class or value for test data.
- Evaluate the accuracy or error of the predictions and analyze the results.

Code:



```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.neighbors import KNeighborsClassifier
8 from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
9
10 # Load the Iris dataset
11 df = pd.read_csv('Iris.csv')
12
13 # Select input features and target variable
14 X = df[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']]
15 y = df['Species']
16
17 # Split the dataset into training and testing data
18 X_train, X_test, y_train, y_test = train_test_split(
19     X,
20     y,
21     test_size=0.2,
22     random_state=42,
23     stratify=y
24 )
25
26 # Standardize the feature values
27 scaler = StandardScaler()
28 X_train = scaler.fit_transform(X_train)
29 X_test = scaler.transform(X_test)
30
31 # Create the K-NN model with K = 5
32 model = KNeighborsClassifier(n_neighbors=5)
```

SHETH L.U.J AND SIR M.V COLLEGE

Artificial Intelligence

Practical No.7

```
pro7.py - C:\Users\gazala\OneDrive\Desktop\pro7.py (3132)
File Edit Format Run Options Window Help
# Split the visualization features into training and testing data
Xv_train, Xv_test, yv_train, yv_test = train_test_split(
    X_visual,
    y_visual,
    test_size=0.2,
    random_state=42,
    stratify=y_visual
)

# Standardize the visualization features
visual_scaler = StandardScaler()

Xv_train = visual_scaler.fit_transform(Xv_train)
Xv_test = visual_scaler.transform(Xv_test)

# Train K-NN using the two visualization features
visual_model = KNeighborsClassifier(n_neighbors=5)
visual_model.fit(Xv_train, yv_train)

# Create a mesh for the decision boundary
x_min = Xv_train[:, 0].min() - 1
x_max = Xv_train[:, 0].max() + 1
y_min = Xv_train[:, 1].min() - 1
y_max = Xv_train[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.arange(x_min, x_max, 0.02),
    np.arange(y_min, y_max, 0.02)
)

# Predict the class for every point in the mesh
Z = visual_model.predict(
    np.c_[xx.ravel(), yy.ravel()]
)

classes[0]: 0,
classes[1]: 1,
classes[2]: 2
]

Z = np.array([class_to_number(value) for value in Z])
Z = Z.reshape(xx.shape)

# Plot the K-NN decision boundary
plt.figure(figsize=(9, 6))

plt.contourf(xx, yy, Z, alpha=0.3)

# Convert training labels into numeric values
train_numbers = np.array([
    class_to_number[value] for value in yv_train
])

plt.scatter(
    Xv_train[:, 0],
    Xv_train[:, 1],
    c=train_numbers,
    edgecolors='black',
    label='Training Data'
)

plt.xlabel('Standardized Petal Length')
plt.ylabel('Standardized Petal Width')
plt.title('K-NN Decision Boundary (K = 5)')
plt.legend()

plt.show()
```

SHETH L.U.J AND SIR M.V COLLEGE

Artificial Intelligence

Practical No.7

Output:

```
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb 4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
----- RESTART: C:/Users/gazala/OneDrive/Desktop/pno.7.py -----
K-NN Accuracy: 0.9333333333333333
Accuracy Percentage: 93.33333333333333 %
Prediction Error: 0.06666666666666665
Error Percentage: 6.666666666666665 %

Confusion Matrix:
[[10  0  0]
 [ 0 10  0]
 [ 0  2  8]]

Classification Report:
              precision    recall  f1-score   support

 Iris-setosa       1.00        1.00        1.00         10
 Iris-versicolor  0.83        1.00        0.91         10
 Iris-virginica    1.00        0.80        0.89         10

 accuracy          0.94
 macro avg         0.93
 weighted avg      0.93
```

SHETH L.U.J AND SIR M.V COLLEGE

Artificial Intelligence

Practical No.7

